



# **Application-Specific Computer Architecture**

**(ELL782,ELL7282)**

**Tanmoy Chakraborty  
Professor, IIT Delhi**

# Logistics

Regular lecture slot: Mon, Thu, 8-9:30 am

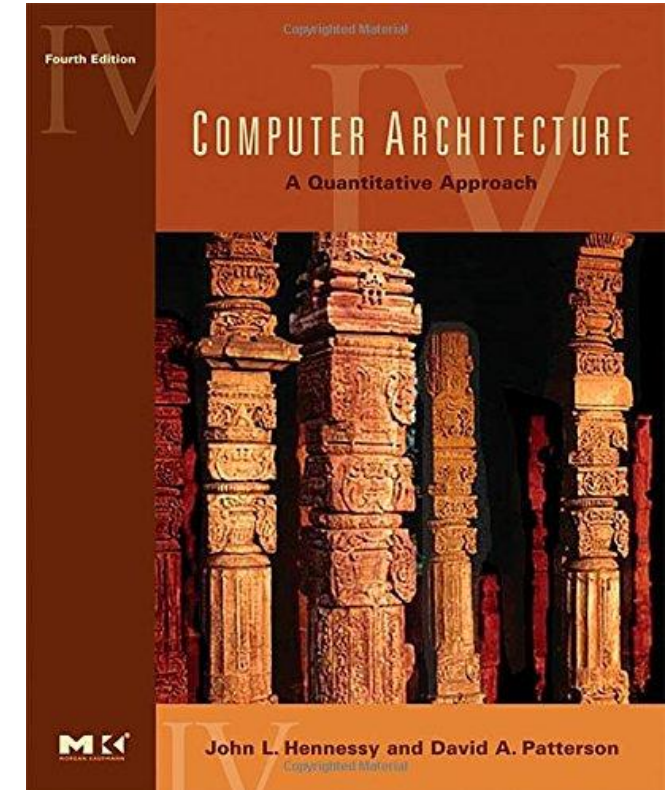
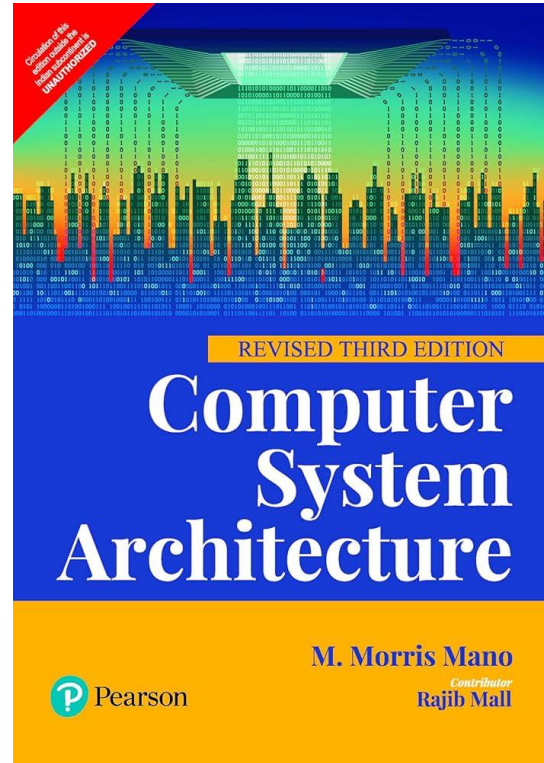
Venue: II 336

TAs: Raj Singh Tomar, Indraayudh Talunkar

Course website: <https://lcs2-iitd.github.io/ELL7282-2601/>

Piazza Link: [https://piazza.com/iit\\_delhi/fall2026/ell7282ell782](https://piazza.com/iit_delhi/fall2026/ell7282ell782)  
(Code: carch2601)

# Textbooks



Slides are primarily  
adopted from this book

# Tentative Syllabus

Introduction

Language of Bits

Assembly Languages

Digital Logic

Computer Arithmetic

Processor Design

Pipelining

Memory System

Multiprocessor Systems \*

I/O and Storage Devices \*

\*If time permits

# Tentative Grading Scheme

- Assignment 1 (10 marks) – Before Midsem
  - Assignment 2 (10 marks) – After Midsem
  - Four Quizzes (20 marks) – Two before Midsem, two after Midsem
  - Daily Quiz: 5 marks
  - Attendance: 5 marks
  - Midsem: 25 marks)
  - Endsem: 25 marks)
- 
- Attendance: 75% (to be eligible to appear in the exam)

# What is Computer Architecture ?

Answer : It is the study of computers ?

- \* Computer Architecture
  - \* The view of a computer as presented to software designers
- \* Computer Organization
  - \* The actual implementation of a computer in hardware.

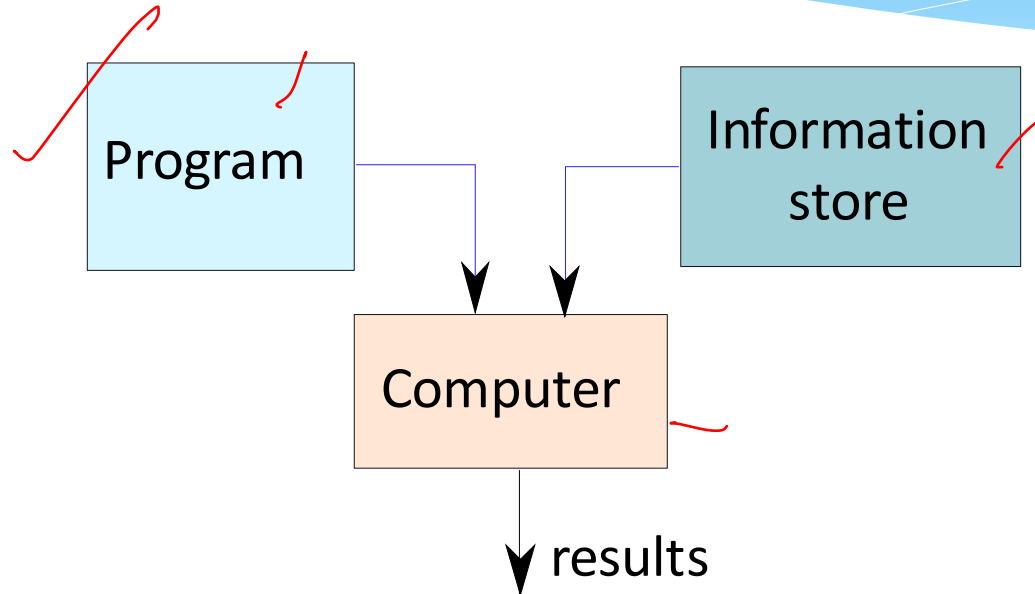
# What is a Computer ?



A computer is a general purpose device that can be **programmed** to **process** information, and yield **meaningful** results.



# How does it work ?



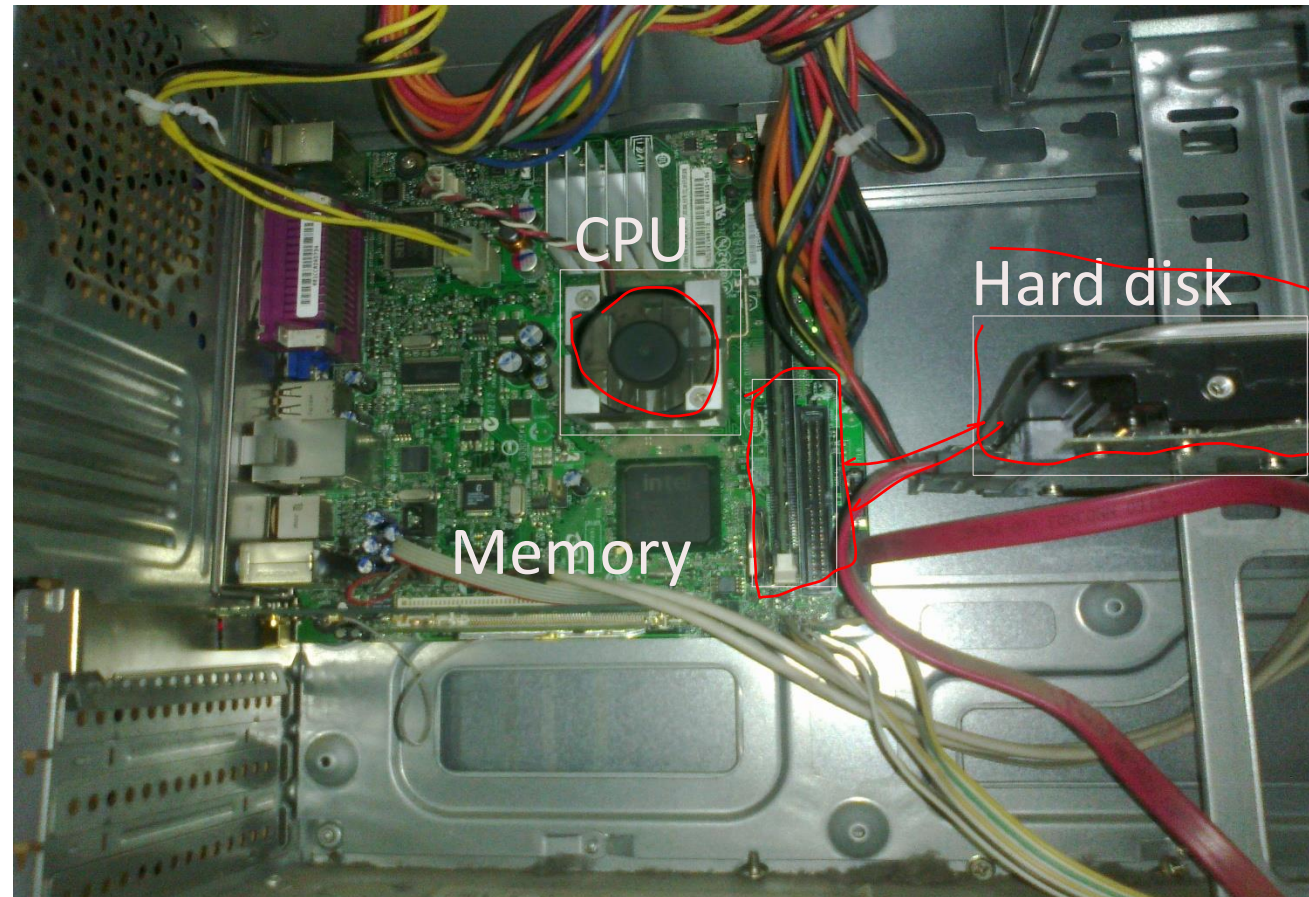
- \* Program – List of instructions given to the computer
- \* Information store – data, images, files, videos
- \* Computer – Process the information store according to the instructions in the program

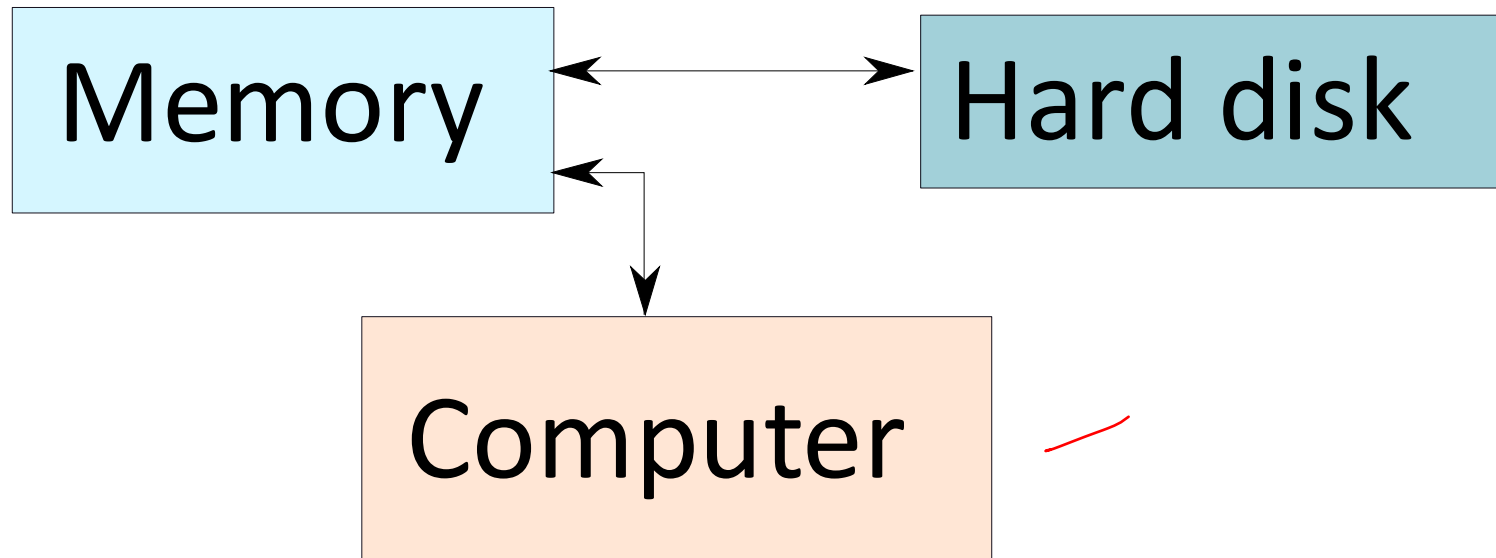


# What does a computer look like ?

- \* Let us take the lid off a desktop computer

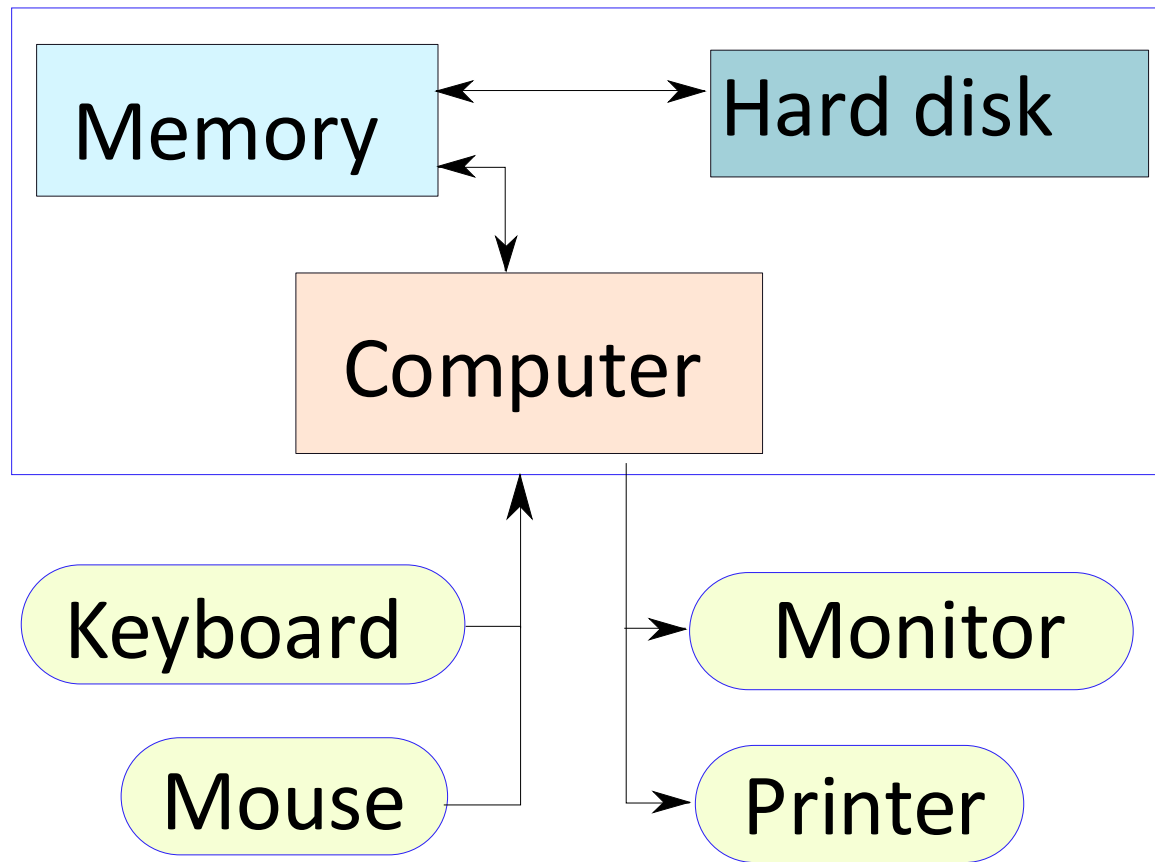
- FPU (Floating Point Unit)
- ISP (Image Signal Processor)
- Microcontrollers (MCU)
- Network Processor / NIC Processor
- Storage Controllers



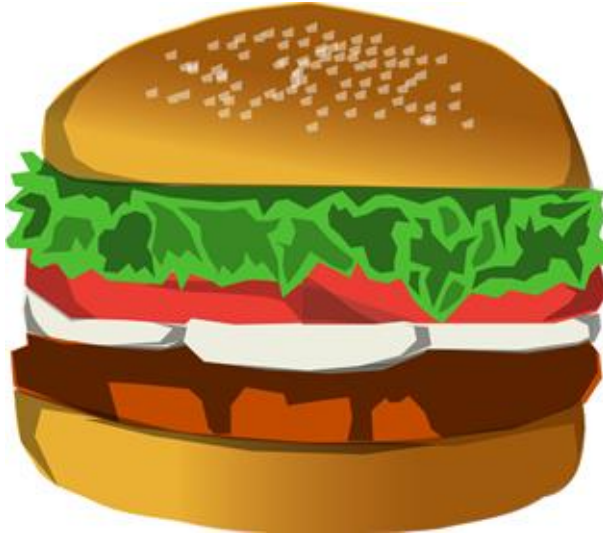


- \* Memory – Stores programs and data. Gets destroyed when the computer is powered off
- \* Hard disk – stores programs/data permanently

# Let us make it a full system ...

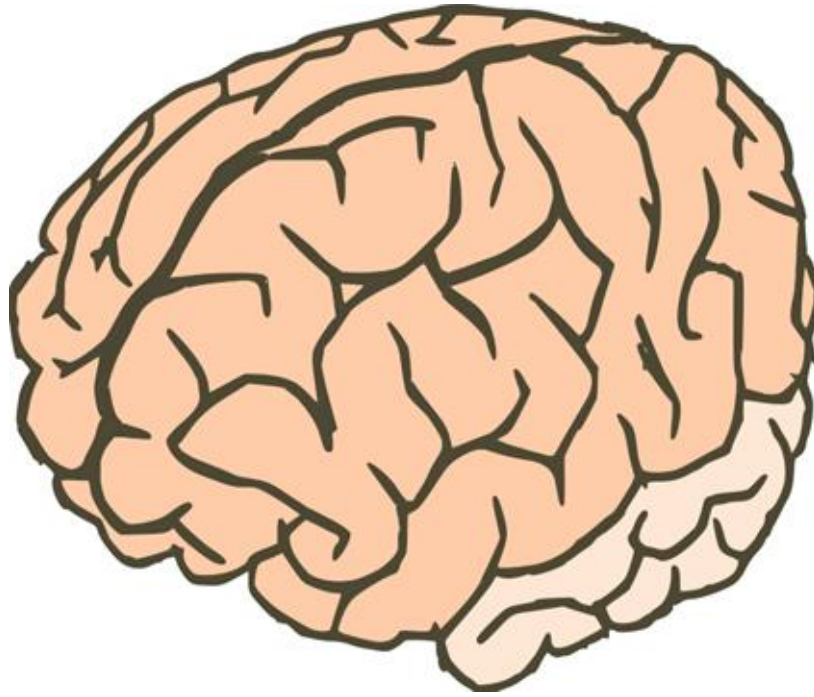


# Food for Thought...



- \* What is the most intelligent computer ?

# Answer ...



\* Our brilliant brains

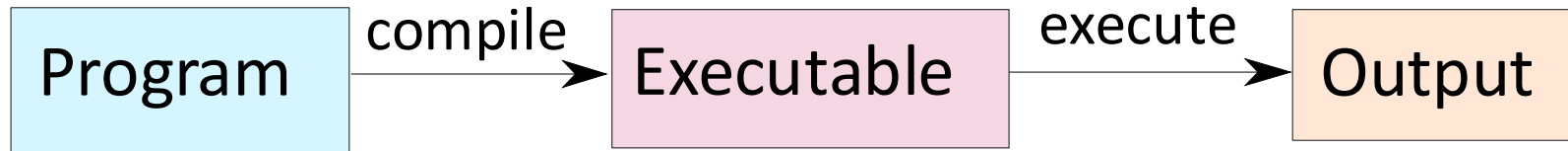
# How does an Electronic Computer Differ from our Brain ?

| Feature                     | Computer   | Our Brilliant Brain |
|-----------------------------|------------|---------------------|
| Intelligence                | Dumb       | Intelligent         |
| Speed of basic calculations | Ultra-fast | Slow                |
| Can get tired               | Never      | After sometime      |
| Can get bored               | Never      | Almost always       |

\* Computers are ultra-fast and ultra-dumb

/

# How to Instruct a Computer ?



- \* Write a program in a high level language – C, C++, Java
- \* **Compile** it into a format that the computer understands
- \* Execute the program

# What Can a Computer Understand ?

- \* Computer can clearly **NOT** understand instructions of the form
  - \* Multiply two matrices
  - \* Compute the determinant of a matrix
  - \* Find the shortest path between Mumbai and Delhi
- \* They understand :
  - \* Add  $a + b$  to get  $c$
  - \* Multiply  $a * b$  to get  $c$



# The Language of Instructions

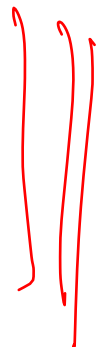
- \* Humans can understand

- \* Complicated sentences
  - \* English, French, Spanish

- \* Computers can understand

- \* Very simple instructions

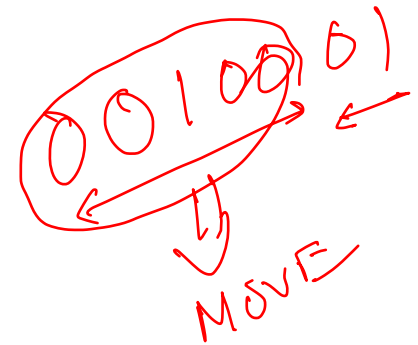
The semantics of all the instructions supported by a processor is known as its instruction set architecture (ISA). This includes the semantics of the instructions themselves, along with their operands, and interfaces with peripheral devices.



# Features of an ISA

- \* Example of instructions in an ISA

- \* Arithmetic instructions : add, sub, mul, div
- \* Logical instructions : and, or, not
- \* Data transfer/movement instructions



- \* Complete

- \* It should be able to implement all the programs that users may write.

# Features of an ISA – II

- \* **Concise**

- \* The instruction set should have a limited size.  
Typically an ISA contains 32-1000 instructions.

- \* **Generic**

- \* Instructions should not be too specialized, e.g.  
add14 (adds a number with 14) instruction is too specialized

- \* **Simple**

- \* Should not be very complicated.

# Designing an ISA

- \* Important questions that need to be answered :
  - \* How many instructions should we have ?
  - \* What should they do ?
  - \* How complicated should they be ?

Two different paradigms : RISC and CISC

RISC  
(Reduced Instruction Set  
Computer)

CISC  
(Complex Instruction  
Set Computer)

# RISC vs CISC

A reduced instruction set computer (**RISC**) implements simple instructions that have a simple and regular structure. The number of instructions is typically a small number (64 to 128). Examples: ARM, IBM PowerPC, HP PA-RISC

A complex instruction set computer (**CISC**) implements complex instructions that are highly irregular, take multiple operands, and implement complex functionalities. Secondly, the number of instructions is large (typically 500+). Examples: Intel x86, AMD, VAX

# CISC

## Who developed it?

CISC emerged from **early computer architects in industry**, most notably at **IBM** and later **Intel**.

## Key contributors:

- IBM engineering teams
- Intel processor architects

## When?

- **1960s–1970s**

## Why CISC?

- Early computers had **very limited memory**
- Goal: **reduce program size** by packing more work into a single instruction
- Instructions were often **complex, multi-cycle, and variable-length**

# RISC

## Who developed it?

RISC was pioneered in **academic and industrial research labs**, led by:

- John Cocke at IBM
- David Patterson at University of California, Berkeley
- John L. Hennessy at Stanford University

## When?

**Late 1970s – early 1980s**

## Why RISC?

- Empirical studies showed most programs use **simple instructions**
- Simpler instructions allow:
  - Pipelining
  - Higher clock speeds
  - Better compiler optimization

| Feature                | CISC                                           | RISC                                             |
|------------------------|------------------------------------------------|--------------------------------------------------|
| Instruction Set        | Large and complex                              | Small and simple                                 |
| Instruction Length     | Variable                                       | Fixed                                            |
| Execution Time         | One instruction may take multiple clock cycles | Most instructions execute in one clock cycle     |
| Number of Instructions | Fewer instructions needed for a task           | More instructions needed for the same task       |
| Hardware Design        | Complex                                        | Simple                                           |
| Control Unit           | Microprogrammed                                | Hardwired                                        |
| Memory Usage           | Efficient (fewer instructions)                 | Less efficient (more instructions)               |
| Power Consumption      | Higher                                         | Lower                                            |
| Performance            | Slower due to complex instruction decoding     | Faster due to simple instructions and pipelining |
| Examples               | Intel x86, AMD x86<br>.....                    | ARM processors, Apple M-series, RISC-V<br>.....  |



# Summary Uptil Now ...

- \* **Computers** are dumb yet ultra-fast machines.
- \* **Instructions** are basic rudimentary commands used to communicate with the processor. A computer can execute billions of instructions per second.
- \* The **compiler** transforms a user program written in a high level language such as C to a program consisting of basic machine instructions.
- \* The **instruction set architecture (ISA)** refers to the semantics of all the instructions supported by a processor.
- \* The instruction set needs to be **complete**. It is desirable if it is also **concise**, **generic**, and **simple**.

# Completeness of an ISA



How can we ensure that an ISA is complete ?

- \* Complete means :
  - \* Can implement all types of programs
  - \* For example, if we just have **add** instructions, we cannot **subtract** (**NOT Complete**)



# Completeness of an ISA – II

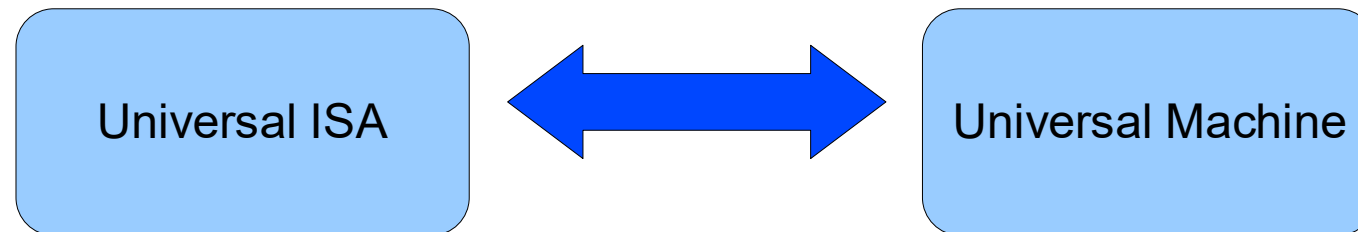


How to ensure that we have just enough instructions such that we can implement every possible program that we might want to write ?

Skip this part

# Answer

- \* Let us look at results in theoretical computer science
- \* Is there an universal ISA ?

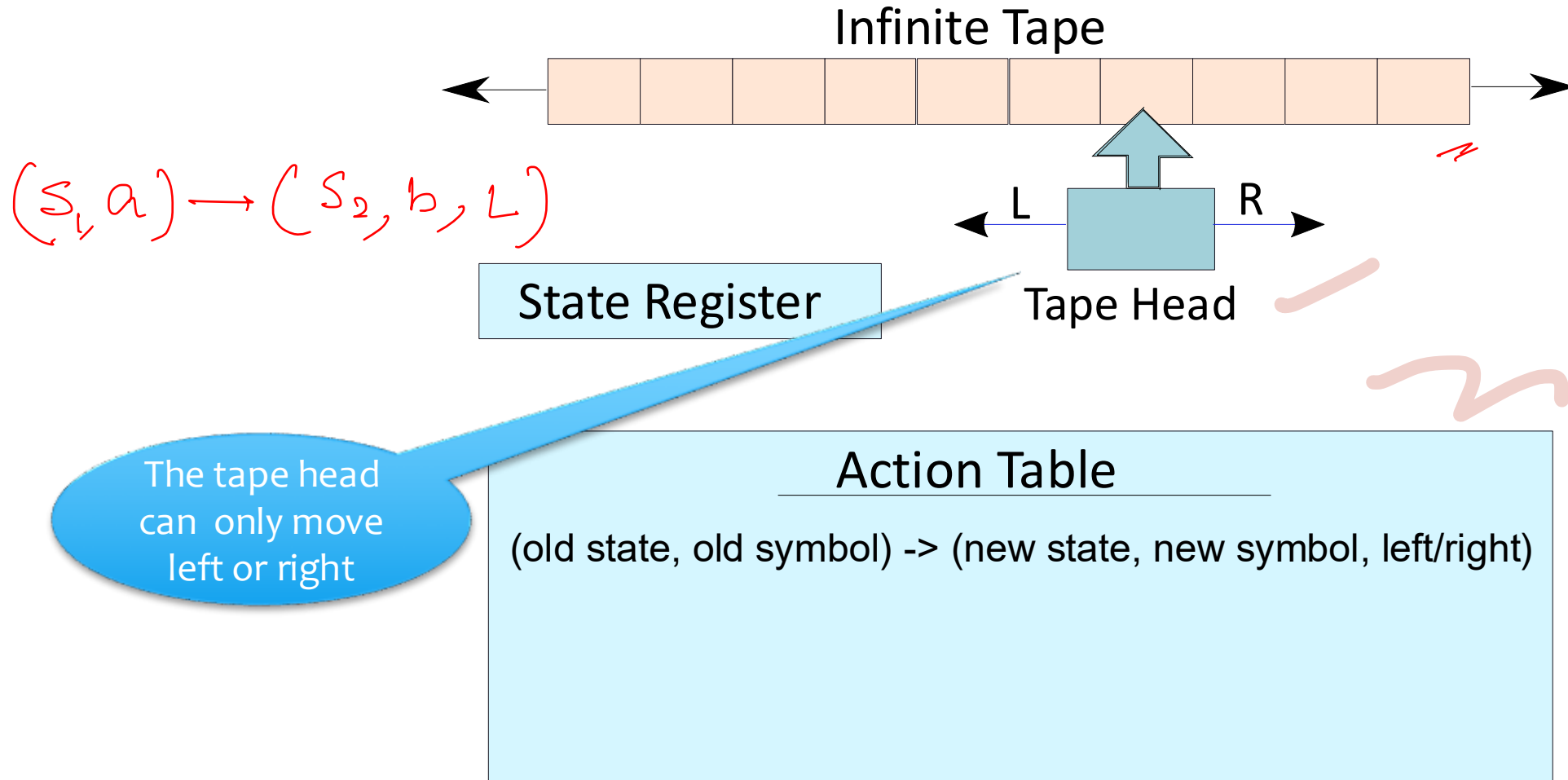


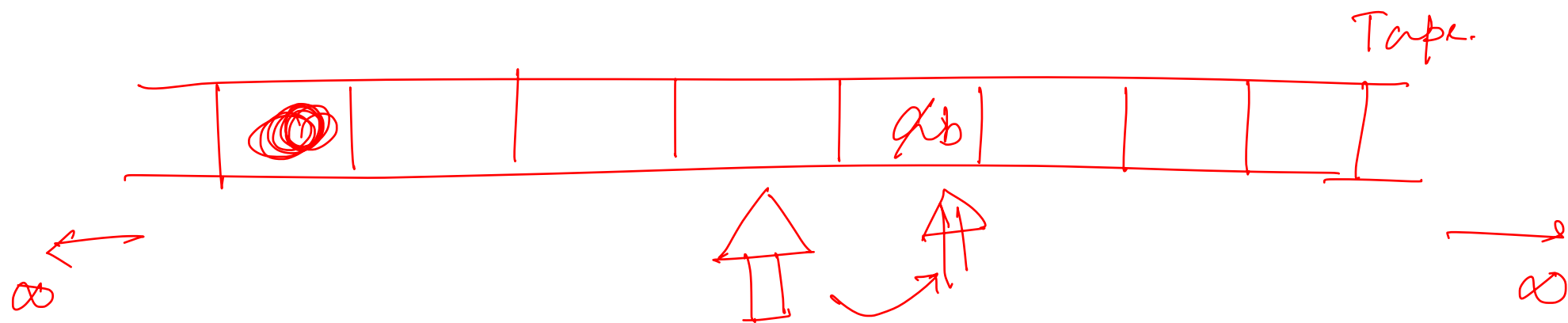
The universal machine has a set of basic actions, and each such **action** can be interpreted as an instruction.

# The Turing Machine – Alan Turing

- \* Facts about Alan Turing
  - \* Known as the father of computer science
  - \* Discovered the Turing machine that is the most powerful computing device known to man
  - \* Indian connection : His father worked with the Indian Civil Service at the time he was born. He was posted in Chhatrapur, Odisha.

# Turing Machine





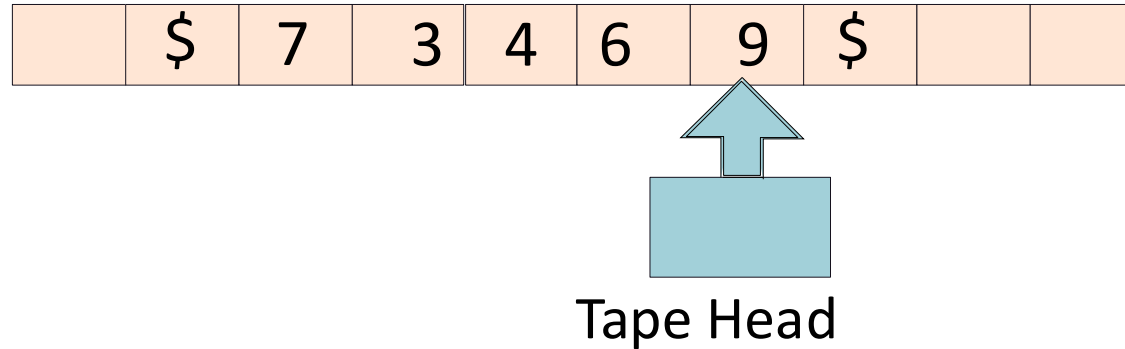
# Operation of a Turing Machine

- \* There is an **inifinite tape** that extends to the left and right. It consists of an infinite number of cells.
- \* The tape head points to a **cell**, and can either move 1 cell to the left or right
- \* Based on the symbol in the **cell**, and its **current state**, the Turing machine computes the transition :
  - \* Computes the **next state**
  - \* Overwrites the symbol in the **cell** (or keeps it the same)
  - \* Moves to the left or right by 1 **cell**
- \* The **action table** records the rules for the transitions.



# Example of a Turing Machine

*Design a Turing machine to increment a number by 1.*



- \* Start from the rightmost position. (state = 1)
- \* If (state = 1), replace a number  $x$ , by  $x+1 \bmod 10$
- \* The new state is equal to the value of the carry
- \* Keep going left till the '\$' sign

# More about the Turing Machine

- \* This machine is extremely simple, and extremely powerful
  - \* We can solve all kinds of problems – mathematical problems, engineering analyses, protein folding, computer games, ...
  - \* Try to use the Turing machine to solve many more types of problems (TODO)

# Church-Turing Thesis

**Church-Turing thesis:** Any real-world computation can be translated into an equivalent computation involving a Turing machine.  
(source: Wolfram Mathworld)

- \* Note : It is a thesis, not a theorem
- \* For the last 60 years, nobody has found a counter-example

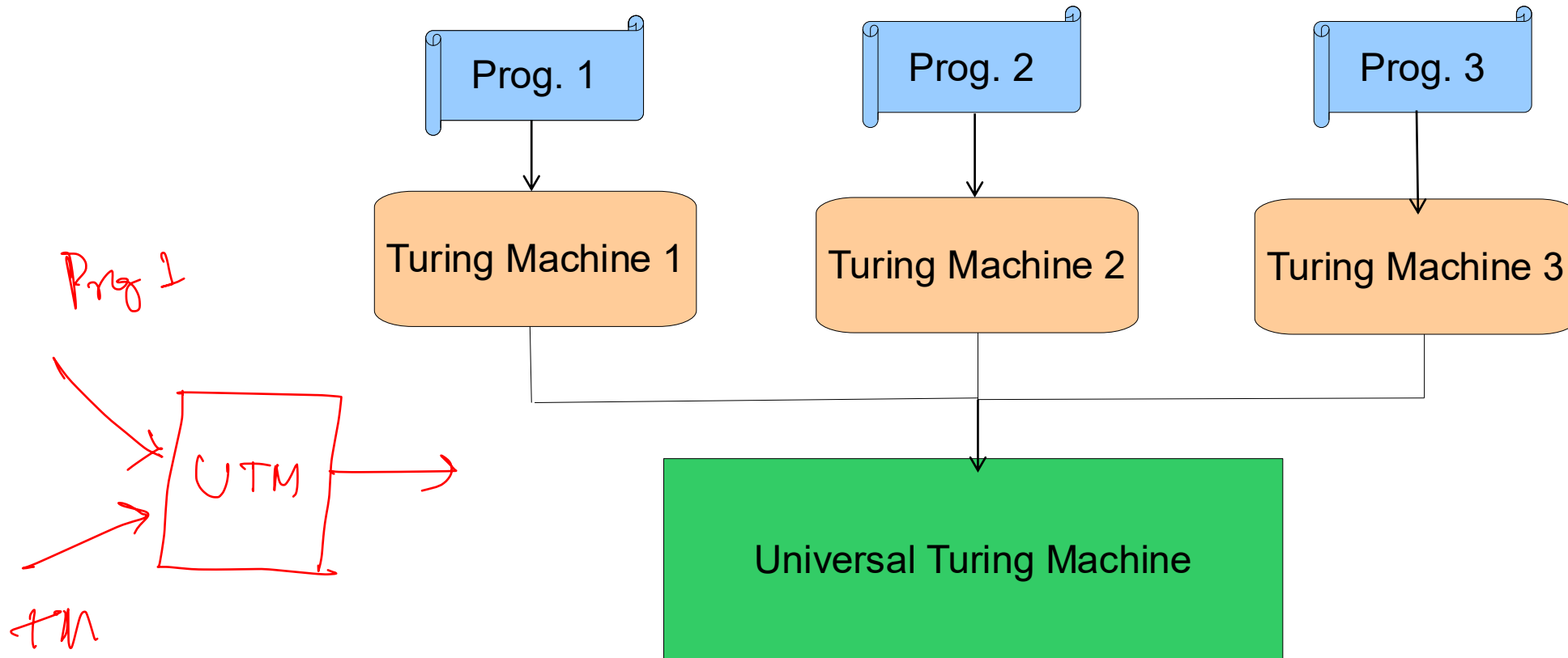
**Definition:**

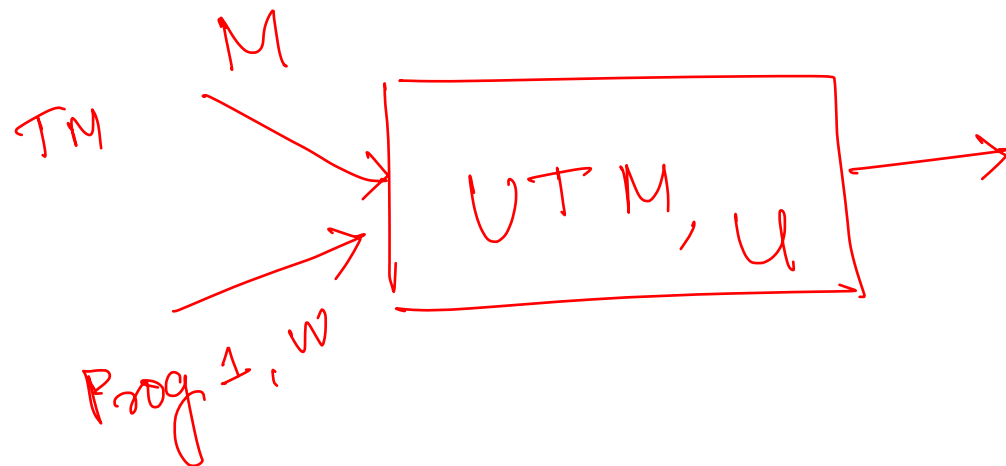
Any computing system that is equivalent to a Turing machine is said to be Turing complete.

# Universal Turing Machine

- \* For every problem in the world, we can design a Turing Machine (Church-Turing thesis)
- \* Can we design a universal Turing machine that can simulate any Turing machine. This will make it a **universal machine (UTM)**
- \* Why not? The logic of a Turing machine is really simple. We need to move the tape head left, or right, and update the symbol and state based on the action table. **A UTM can easily do this.**
- \* A UTM needs to have an action table, state register, and tape that can simulate any arbitrary Turing machine.

# Universal Turing Machine





$$a^n b^n$$

$$L_1 = \{ w \mid \text{All elements are prime no.} \}$$

$$L_2 = \{ \quad \quad \quad ? \}$$

The Language

$$A_{TM} = \{ \langle M, w \rangle \mid M \text{ is a TM and } M \text{ accepts } w \}$$

is Turing Recognizable

Given a description of a TM and an input, can we determine whether the machine accepts it?

—

M accepts  $w$  : our algorithm will halt and Accept  
M rejects  $w$  : our " will halt and reject  
M loops on  $w$  : " " will not halt

UTM

i/p :  $M, w$

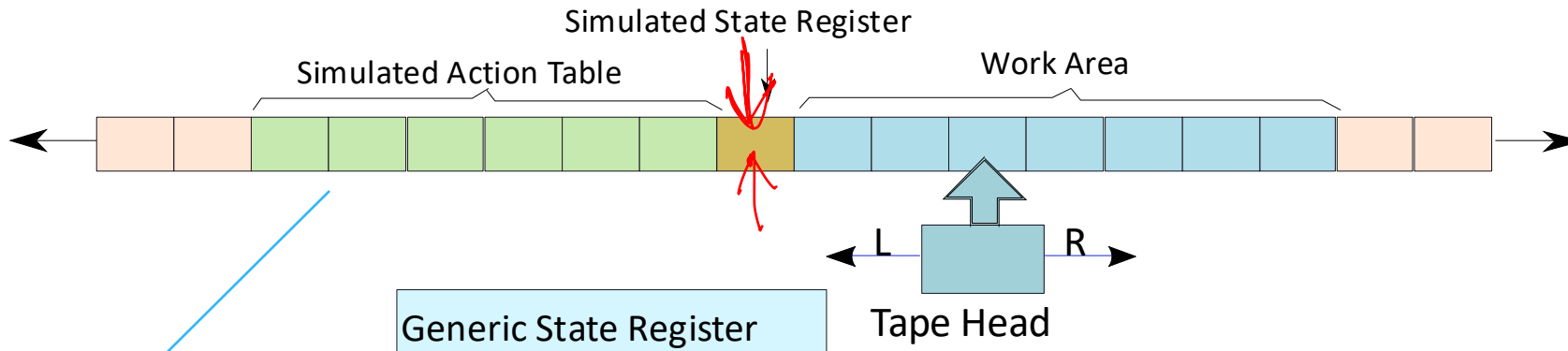
Action: - Simulate  $M$   
- Behave just like  $M$  would (may accept, may reject or loop)

The UTM is a recognizer (not a decider) for

$A_{TM}$

# A Universal Turing Machine

1334 @ 34 @ 0 @ 1335 @ 10 @

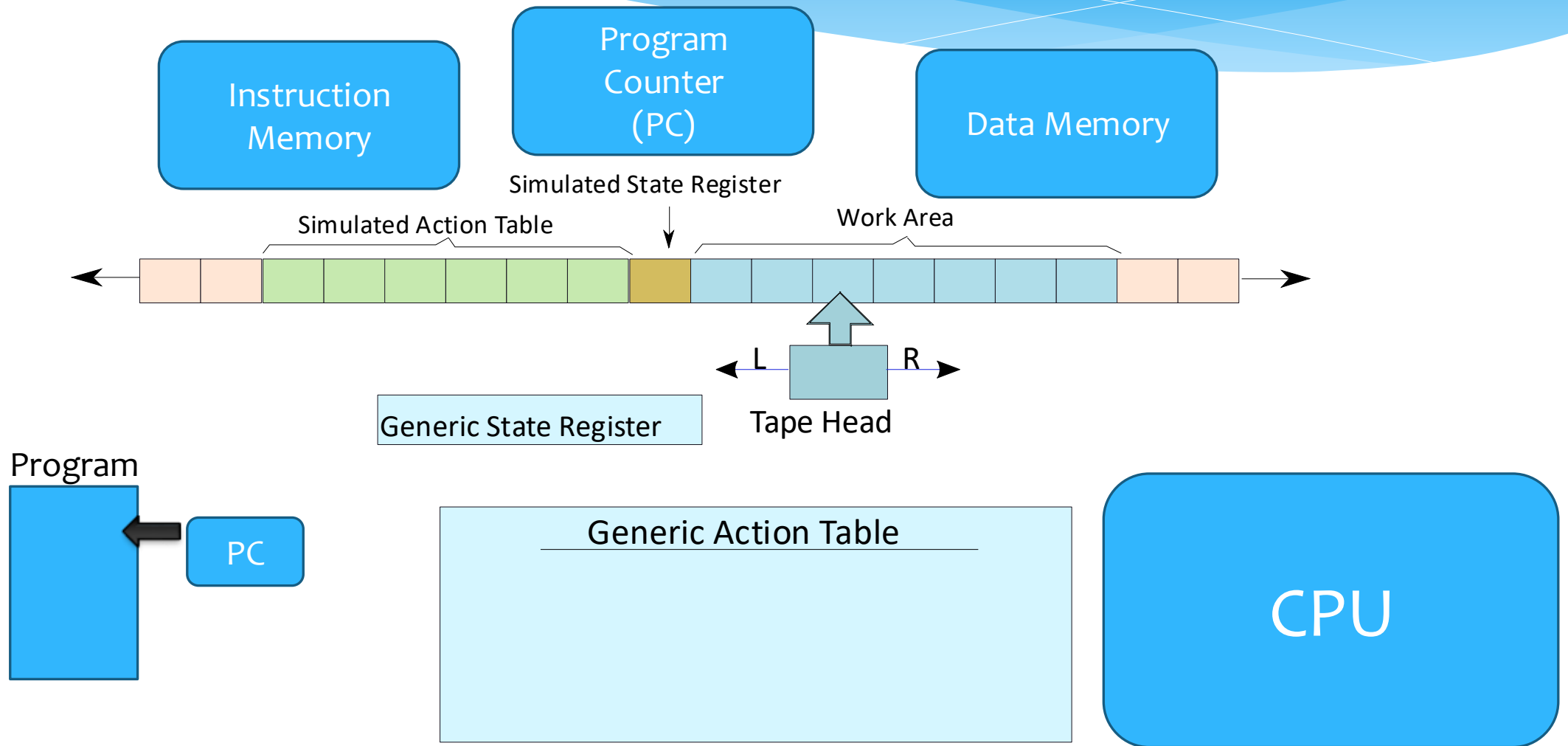


1334@34@0@1335@10@

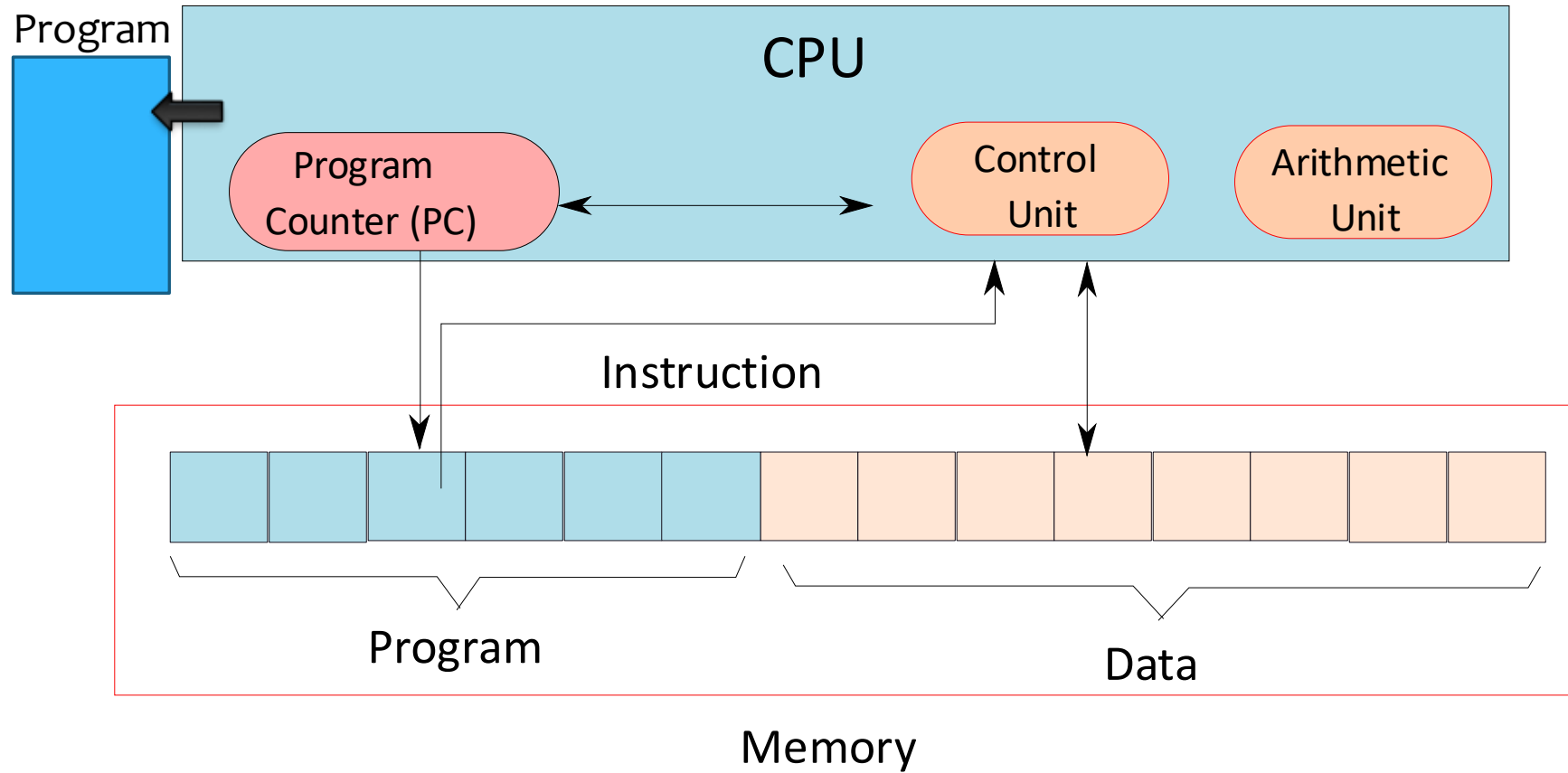
- '@' symbol is the delimiter
- Move from state 1334 to 1335 if symbol 34 is encountered.
- We move left (0), and write a value of 10.



# A Universal Turing Machine - II



# Computer Inspired from the Turing Machine



# Elements of a Computer

- \* Memory (array of bytes) contains
  - \* The program, which is a sequence of instructions
  - \* The program data → variables, and constants
- \* The program counter(PC) points to an instruction in a program
  - \* After executing an instruction, it points to the next instruction by default
  - \* A branch instruction makes the PC point to another instruction (not in sequence)
- \* CPU (Central Processing Unit) contains the
  - \* Program counter, instruction execution units

# Let us now design an ISA ...

- \* Single Instruction ISA
  - \* sbn – subtract and branch if negative
- \* Add (a + b) (assume temp = 0)

*sbn a, b, <line no>*  
*a = a - b*  
*if (a < 0)*  
*goto <line no>*

```
1: sbn temp, b, 2
2: sbn a, temp, exit
```

*temp = temp - b = -b*  
*a = a + b*

*sbn a, b, <line no>*  
*a = a - b*  
*if (a < 0)*  
*goto <line no>*

Add numbers: 1 --- 10

$a = 1$

$index = 10$

$sum = 0$

$temp = 0$

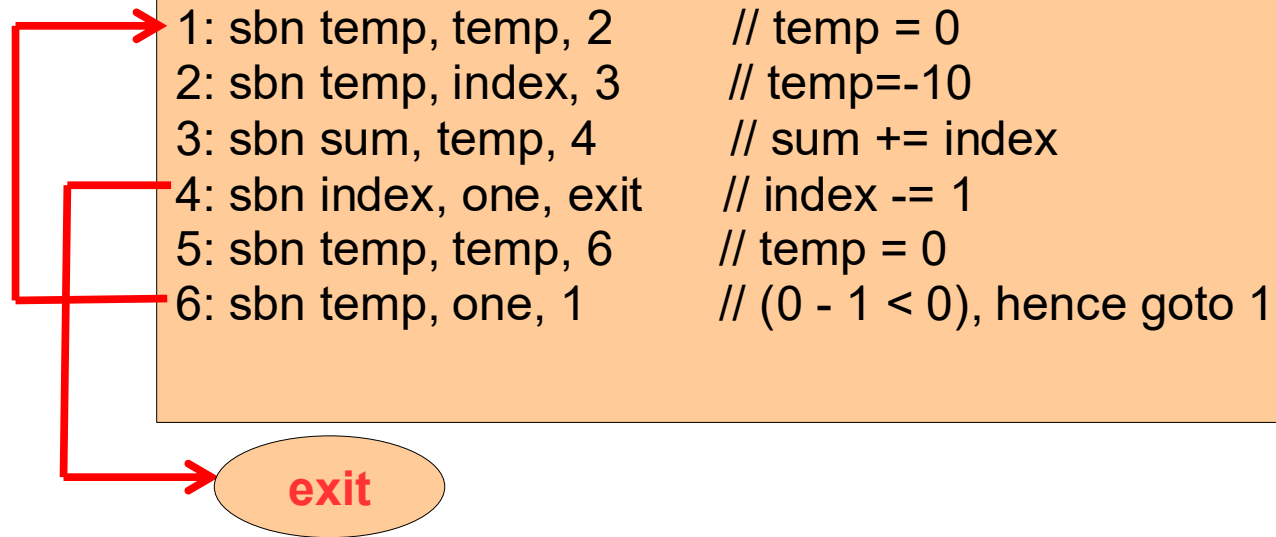
1.  $subn\ temp, temp, 2$  ;  $temp = 0$
2.  $subn\ temp, index, 3$  ;  $temp = -10$
3.  $subn\ sum, temp, 4$  ;  $sum = 10$
4.  $subn\ \underline{index, a, exit}$  ;  $index = 9$
5.  $subn\ temp, temp, 6$  ;  $temp = 0$
6.  $subn\ temp, a, 1$

# Single Instruction ISA - II

\* Add the numbers – 1 ... 10

Initialization:

```
one = 1  
index = 10  
sum = 0
```



# Multiple Instruction ISA

- \* **Arithmetic Instructions**

- \* add, subtract, multiply, divide

- \* **Logical Instructions**

- \* or, and, not

- \* **Move instructions**

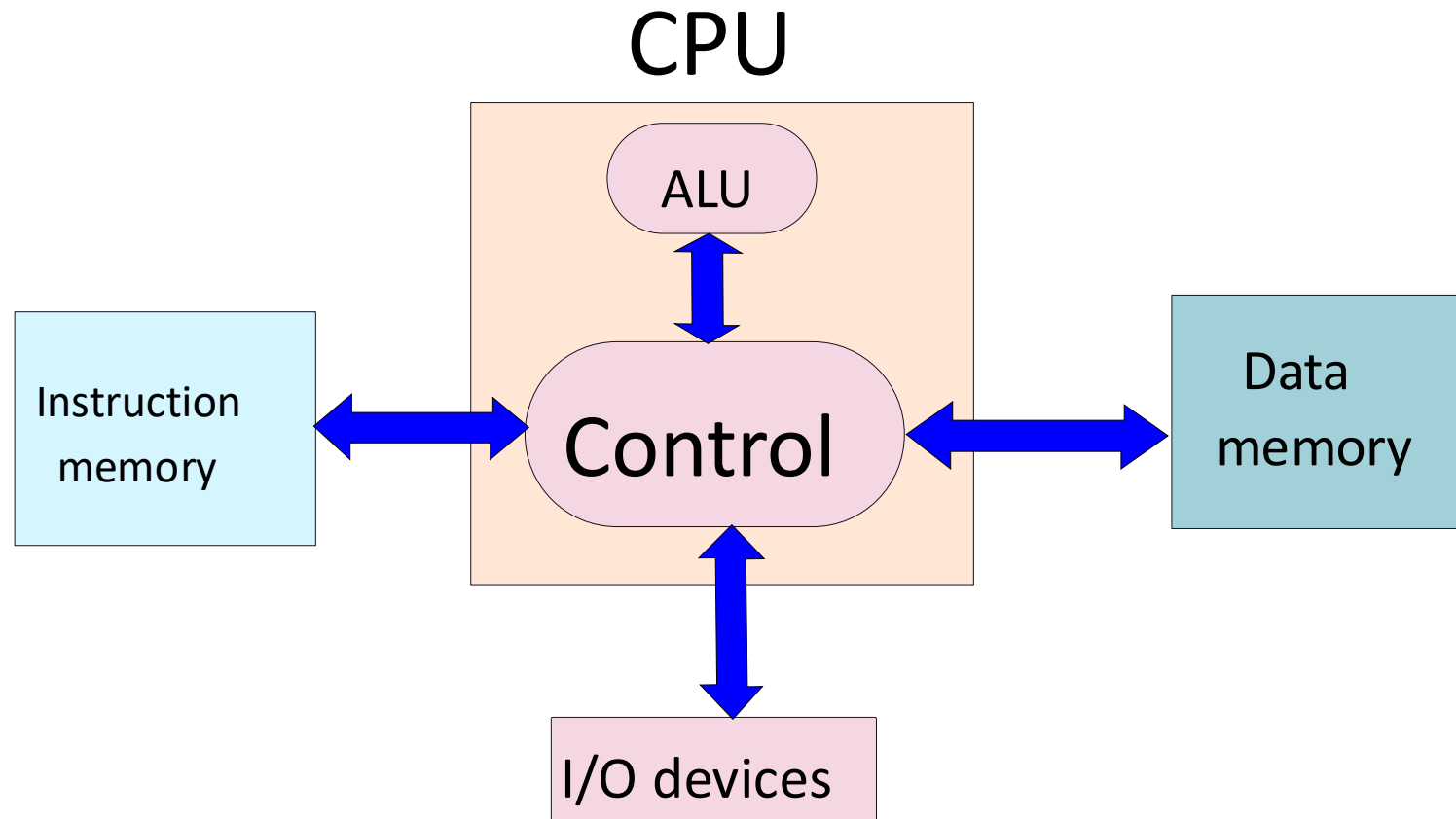
- \* Transfer values between memory locations

- \* **Branch instructions**

- \* Move to a new program location, based on the values of some memory locations

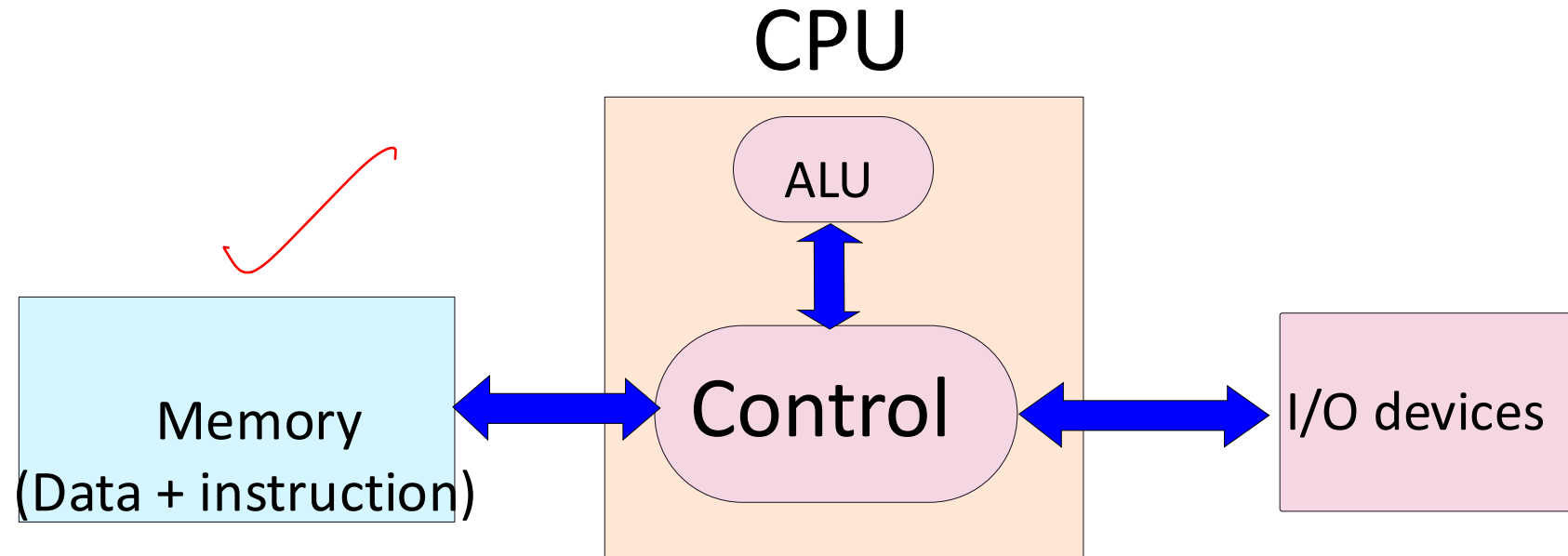
# Designing Practical Machines

Harvard Architecture





# Von-Neumann Architecture



# Problems with Harvard/ Von-Neumann Architectures

- \* The memory is assumed to be one large array of bytes

- \* It is very very **slow**



**General Rule: Larger is a structure, slower it is**

- \* **Solution:**

- \* Have a small array of named locations (**registers**) that can be used by instructions
- \* This small array is very fast



**Insight: Accesses exhibit locality (tend to use the same variables frequently in the same window of time)**

# Uses of Registers

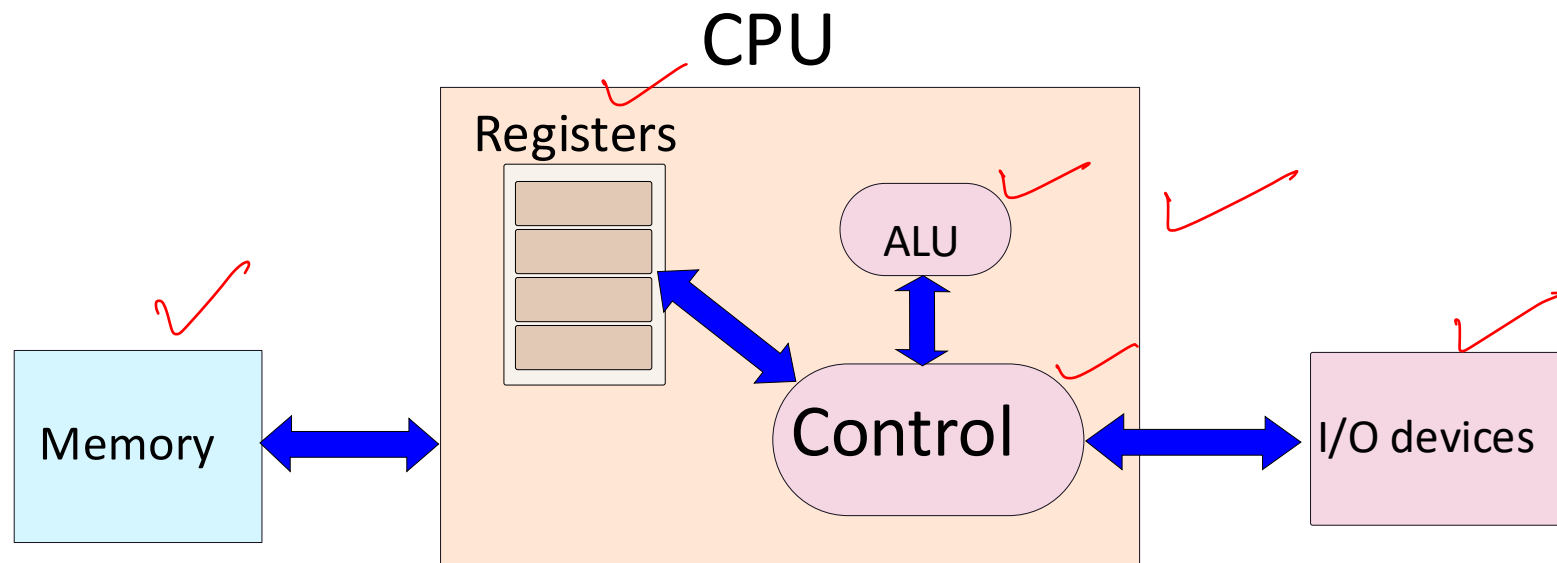
- \* A CPU (Processor) contains set of registers (16-64)
- \* These are named storage locations.
- \* Typically values are loaded from memory to registers.
- \* Arithmetic/logical instructions use registers as input operands
- \* Finally, data is stored back into their memory locations.

# Example of a Program in Machine Language with Registers

```
1: r1 = mem[b] // load b
2: r2 = mem[c] // load c
3: r3 = r1 + r2 // add b and c
4: mem[a] = r3 // save the result
```

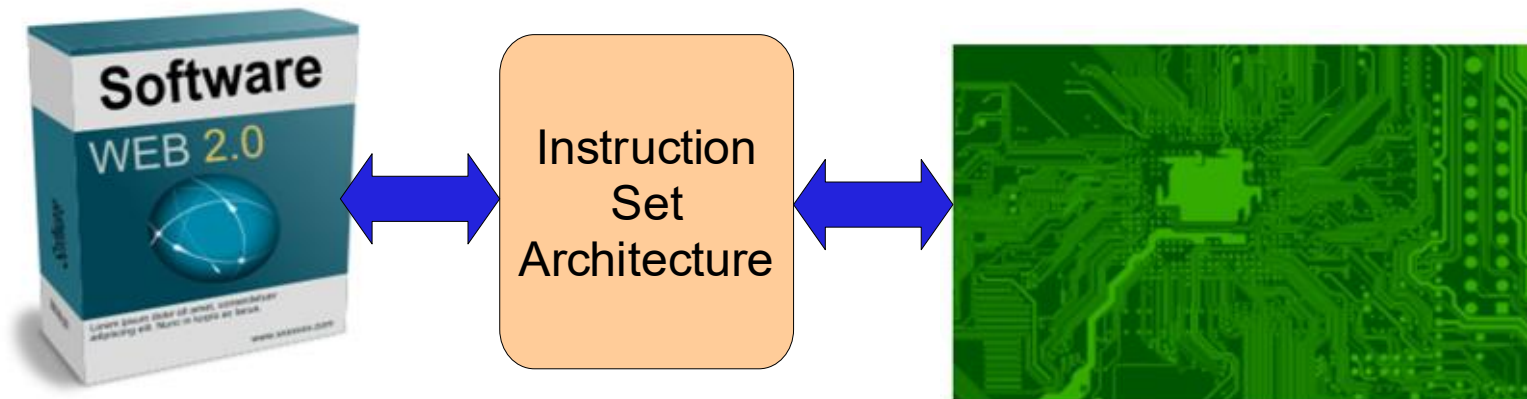
- \* r1, r2, and r3, are registers
- \* mem → array of bytes representing memory

# Machine with Registers



# Where are we ...

- \* We have derived the structure of a computer from theoretical fundamentals.
- \* It has a **CPU** with a program counter & registers, memory, and peripherals.
- \* The **Instruction Set Architecture (ISA)** is the link between **hardware** and **software**.

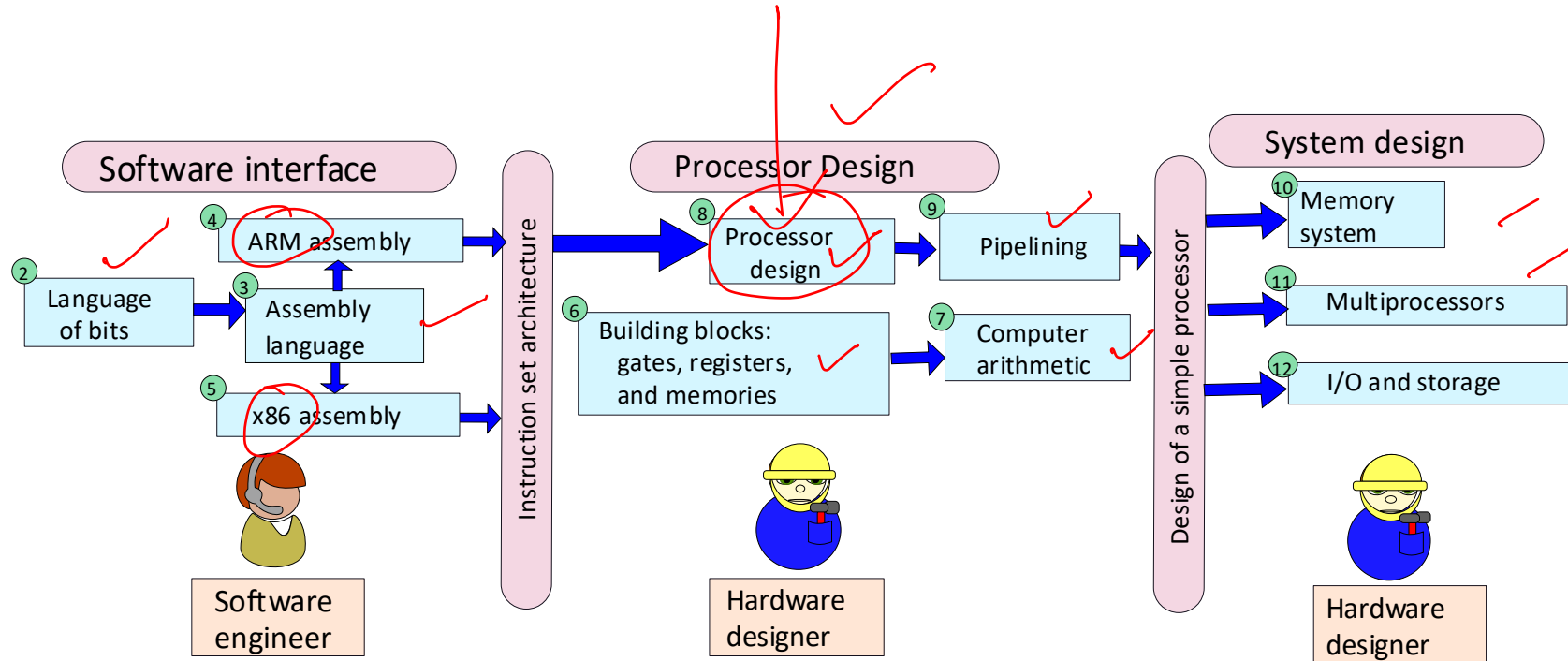


# Instruction Set Architecture

- \* Interface between **software** and **hardware**
  - \* A **compiler** converts a program into machine instructions in the given ISA
  - \* The processor executes the instructions in the ISA
- \* We shall first look at the software aspect of the ISA (**assembly programs**)
- \* Then look at implementing the ISA by designing the **processor**
- \* Then, we shall make the computer more efficient by designing fast **memory/ storage** systems
- \* At the end, we will look at **multiprocessors**

# Roadmap of the Course

Simple RISC







THE END